AM55 - Mesos

Network Protocol Documentation

Version 1.0 - June 2026

Authors: Di Girolamo Daniele, Diana Nicolo, Bontempi Francesco, Cagnazzo Giacomo

Scope

This document provides a concise description of the AM55 client-server network protocol, with a focus on object-oriented abstractions, programming by contract, RMI/Socket equivalence, heartbeat management, and the command/message communication cycle.

Technical baseline

The document describes the interfaces and methods actually used by the network protocol. The discussion is intentionally kept at a high level: it includes public interfaces and communication flows, while omitting low-level implementation details.

Contents

1. Architectural overview

2. Main protocol contracts

3. Client-server connection

4. Sending commands: Command Pattern

5. Messages, ClientModel update and delivery

6. Asynchronous RMI and equivalence with Socket

7. Ping, timeout and alive checkers

1. Architectural overview

The AM55 network protocol implements client-server communication based on commands and messages. The client never executes operations directly on the game model. Instead, it sends a command object to the server through the receiveCommand(ServerCommand, VirtualView) method exposed by the VirtualServer interface. The server executes the request on its GameController/GameModel and then notifies clients through MessageToClient objects delivered via the onMessage(MessageToClient) method exposed by VirtualView.

The central design decision is to make client and server communicate through interfaces that are independent from the concrete network technology. Therefore, the same logical operations are used with both RMI and Socket: only the transport adapter changes, while client and server application logic remains unchanged. In particular, ClientImpl stores a generic field named server of type VirtualServer. With RMI, this reference is the remote proxy obtained from the registry; with Socket, it is a local ServerStub that serializes command objects over TCP. In both cases, the client application code perceives a single logical server and can behave as if the network were not present.

The protocol is intentionally asynchronous at application level, both on the client side and on the server side, so that RMI and Socket can share the same application semantics.

2. Main protocol contracts

Element	Contractual responsibility	Main methods
VirtualServer	Exposes the methods that the client can invoke on the server.	receiveCommand(ServerCommand command, VirtualView sender) close()
VirtualView	Exposes the methods that the server can invoke on the client.	onMessage(MessageToClient message) getPlayerId() setPlayerId(String playerId) close()
ServerCommand	Client-server command contract. Each command is serializable and declares whether it requires synchronization.	requiresLock() execute(ServerApplication, VirtualView)
MessageToClient	Server-client message contract. Each message defines how it updates the client model, how it is delivered, and whether it triggers client-side network actions.	update(ClientModel) deliver(String, MessageDelivery) executeClientNetworkAction(ClientConnectionControl) shouldUpdateModel()
MessageDelivery	Server-side delivery strategy.	sendTo(...) broadcast(...) sendToSession(...) broadcastToLobby(...)
ClientConnectionControl	Client-side contract for network actions triggered by messages: ping, pong and controlled shutdown.	startPing() pongFromSever() closeConnection()
ClientModelUpdater	Interface that handles updates produced by network messages on the client-side model.	handleUpdate(MessageToClient message)

3. Client-server connection

The connection phase is designed to preserve the same logical contract for both RMI and Socket, while relying on different infrastructural blocks.

RMI

- The server exports ServerApplication as a remote object implementing VirtualServer and makes it available through the RMI registry.
- The client obtains a VirtualServer reference from the registry: effectively, a proxy/stub generated and managed by the RMI runtime.
- ClientImpl extends UnicastRemoteObject and implements VirtualView, so it can be passed to the server as a callback endpoint.
- Stub and skeleton are not visible application classes in RMI: they are provided by the runtime, which manages marshalling, unmarshalling and remote call dispatch.

Socket

- SocketServer opens a ServerSocket, accepts TCP connections and creates one ClientSkeleton for each client.
- ClientSkeleton implements VirtualView on the server side: it reads ServerCommand objects from the stream, forwards them to ServerApplication.executeCommand(command, this), and writes MessageToClient objects back to the client.
- On the client side, ServerStub implements VirtualServer: it writes serialized commands to the ObjectOutputStream and runs a listener that reads messages from the server.

ClientImpl always uses the server field typed as VirtualServer. For this reason, the UI and the client controller do not distinguish whether the selected network connection is RMI or Socket. Network infrastructure details can therefore be replaced without changing the application logic.

4. Sending commands: Command Pattern

Requests from the client to the server are sent using the Command design pattern. Each game or network operation is represented by a concrete class implementing ServerCommand, such as RegisterLobbyCommand, CreateGameCommand, JoinGameCommand, PlaceTotemCommand, PickCardCommand, PickSpecialCommand, QuitGameCommand, QuitLobbyCommand and PingCommand.

This design makes it possible to send specific, typed objects over the network. The network layer is only responsible for serializing and deserializing the command object; the command itself encapsulates the request parameters and the behavior to be executed on the server through execute(ServerApplication, VirtualView).

The ServerCommand interface also includes requiresLock(). This method explicitly declares whether the command mutates the shared game model and must therefore be executed under gameLock. Game commands such as create, join, place totem and pick card require the lock; PingCommand does not, because it only updates heartbeat state and does not modify the game model.

5. Messages, ClientModel update and delivery

The result of a command is not returned directly to the caller. After execution on the GameController, the server produces one or more MessageToClient objects. Each message contains both the logic needed to update the ClientModel and the strategy used to deliver it: unicast to one player, broadcast to the whole game, unicast to a lobby session, or broadcast to the lobby.

On the client side, VirtualView.onMessage(MessageToClient) is the entry point for server-client notifications. ClientImpl delegates processing to executeUpdate(message). If message.shouldUpdateModel() returns true, the client enters a synchronized section on the model and invokes model.handleUpdate(message), the method exposed

by the ClientModelUpdater interface. In this way, the model remains the sole owner of its internal state, while the message encapsulates the delta or snapshot to be applied.

Network messages such as StartPingMessage and PongMessage override shouldUpdateModel() by returning false. They do not update the game model; instead, they invoke executeClientNetworkAction(ClientConnectionControl) to start pinging or record the reception of a pong. The separation between application updates and technical network actions prevents both the UI and the model from depending on network details.

Message	Main effect	Delivery
ErrorMessage	Notifies a single client about an error, updates lastError/stateRequest and signals that create/join did not complete successfully.	sendTo(playerId)
GameBroadcastInfo	Game-level informational message; clears possible errors and updates textual state without replacing the game view.	broadcast()
GameCrashedBroadcast	Notifies abnormal game termination, marks the client model as ended/crashed and closes the client connection.	broadcast() + closeConnection()
GameEndResolveMessage	Carries the final snapshot and end-game result; updates endGameResultView and marks the game as completed.	broadcast()
LobbyStatusMessage	Updates the lobby view for waiting clients and keeps the client in lobby state.	broadcastToLobby()
MultipleMessages	Composite message: applies and delivers all contained MessageToClient objects in order.	delegated to contained messages
PickCardMessage	Updates the picked card, player food/PP, currentPlayer and current game state.	broadcast()
PlaceTotemMessage	Updates totem placement, currentPlayer and current game state.	broadcast()
PongMessage	Technical heartbeat response; does not update the ClientModel and records the received pong on the client side.	sendTo(playerId) + pongFromSever()
QuitGameMessage	Notifies voluntary game shutdown, updates the last available snapshot and closes the client connection.	broadcast() + closeConnection()
QuitLobbyMessage	Notifies lobby exit or lobby shutdown; updates lobby state and closes the client connection.	sendToSession(sessionId) or broadcastToLobby() + closeConnection()
StartPingMessage	Technical message that enables client-side heartbeat; it does not update the ClientModel.	sendToSession(sessionId) + startPing()
UpdateViewMessage	Carries an updated game snapshot and moves the client out of the lobby state.	broadcast()
WaitingMessage	Informs a single player that the game is waiting for additional participants and updates the waiting state.	sendTo(playerId)

6. Asynchronous RMI and equivalence with Socket

Socket transport is naturally asynchronous: the client writes a command to a stream and receives messages through an independent listener thread; the server reads commands from a ClientSkeleton and sends responses through an output stream. To avoid different semantics between Socket and RMI, RMI is also made asynchronous at application level.

In ServerApplication.receiveCommand(command, sender), the remote RMI call does not execute the command directly inside the RMI dispatch thread: it submits the command to an ExecutorService.

Similarly, ClientImpl.onMessage(message) submits the update to a client-side executor. Therefore, RMI does not behave as a regular synchronous call with an immediate response, but as an event-driven channel equivalent to Socket.

7. Ping, timeout and alive checkers

In accordance with the project specification, the group implemented a ping mechanism to detect player disconnections both in the lobby and during the game.

The sequence starts when the client registers in the lobby: the server stores the client VirtualView, initializes its timestamp in lastPingByClient, sends StartPingMessage and starts the server-side alive checker.

StartPingMessage does not update the model; it calls ClientConnectionControl.startPing(). From that moment on, ClientImpl sends a PingCommand every 1500 ms through a dedicated thread.

PingCommand does not require a lock and invokes ServerApplication.ping(sender), which updates lastPingByClient, namely the map that stores each VirtualView together with the instant at which its last ping reached the server, and replies with PongMessage. PongMessage does not update the model: it calls pongFromSever(), allowing the client to store the timestamp of the last pong received.

Side	Mechanism	Value	Equivalent missed heartbeats
Client	Periodic PingCommand sending	PING_INTERVAL_MS = 1500 ms	1 ping every 1.5 seconds
Server	Maximum time without receiving a ping from the client	PING_TIMEOUT_MS = 6000 ms	The server considers the client disconnected after 4 missed pings.
Client	Maximum time without receiving a pong from the server	SERVER_TIMEOUT_MS = 8000 ms	The client considers the server disconnected after about 5 missed pings.
Server and client	Alive checker period	1500 ms	Check frequency aligned with heartbeat transmission.

Alive checkers are supervision timers. The server-side alive checker periodically scans lastPingByClient, removes expired clients and delegates disconnection handling to a dedicated executor.

If the disconnected client was only in the lobby and all other connected users are also in the lobby, only the session of the disconnected client is closed, while the other clients remain active.

If the client was in a game and there are also players in the lobby, the server triggers the game-crash flow, notifies both in-game and lobby clients so that they disconnect, and then resets itself to accept new players and allow a new game.

The same approach is applied when all clients are in the game, even if the game is still in the startup phase: the game terminates, all players are notified through the appropriate message and then disconnect. Afterwards, the server resets itself to accept new players and allow a new game.

The client-side alive checker verifies that the server keeps replying with PongMessage. If the timeout is exceeded, the client stops pinging, closes the connection and, only in the Socket case, actually closes the server abstraction because it is the ServerStub. The client process then terminates.

Conclusion

The AM55 protocol implements a model based on asynchronous commands and polymorphic messages, clearly separating the game domain from the network transport. The combination of VirtualServer, VirtualView, ServerCommand, MessageToClient and ClientModelUpdater makes the architecture extensible, testable and consistent with object-oriented design and programming by contract.